SYNAPSE Security and Compliance Documentation

Security Team

March 7, 2026

Contents

1 Introduction **2**
1.1 Security Philosophy . . . 2
1.2 Scope of Security Coverage . . . 2
1.3 Security Definitions . . . 2
1.3.1 OWASP (Open Web Application Security Project) . . . 2
1.3.2 CVE (Common Vulnerabilities and Exposures) . . . 2
1.3.3 CVSS (Common Vulnerability Scoring System) . . . 3
1.3.4 PII (Personally Identifiable Information) . . . 3
1.3.5 JWT (JSON Web Token) . . . 3
1.3.6 RBAC (Role-Based Access Control) . . . 3
1.3.7 STRIDE Threat Model . . . 3
1.4 Regulatory Context . . . 3

2 Threat Model **4**
2.1 Assets to Protect . . . 4
2.1.1 User Data . . . 4
2.1.2 API Keys and Secrets . . . 4
2.1.3 Scraped Content . . . 4
2.1.4 AI Models and Embeddings . . . 4
2.1.5 Infrastructure . . . 5
2.2 Threat Actors . . . 5
2.2.1 Unauthenticated Users . . . 5
2.2.2 Authenticated Users . . . 5
2.2.3 Admin Users . . . 5
2.2.4 External Attackers . . . 5
2.2.5 Insiders . . . 5
2.3 Attack Vectors . . . 5
2.3.1 Web-Based Attacks . . . 5
2.3.2 API-Based Attacks . . . 5
2.3.3 Network-Based Attacks . . . 6
2.3.4 Supply Chain Attacks . . . 6
2.3.5 Social Engineering . . . 6
2.4 STRIDE Threat Model with Mitigations . . . 6
2.5 Risk Matrix: Top 15 Threats . . . 6

3 OWASP Top 10 (2021) Coverage **8**
3.1 A01: Broken Access Control . . . 8
3.1.1 Description . . . 8
3.1.2 Risk in SYNAPSE Context . . . 8
3.1.3 Mitigation Implemented . . . 8
3.1.4 Code Example: Django Permission Classes . . . 8
3.2 A02: Cryptographic Failures . . . 9

3.2.1 Description . . . 9
3.2.2 Risk in SYNAPSE Context . . . 9
3.2.3 Mitigation Implemented . . . 9
3.2.4 Code Example: Django Encrypted Fields . . . 9
3.3 A03: Injection . . . 10
3.3.1 Description . . . 10
3.3.2 Risk in SYNAPSE Context . . . 10
3.3.3 Mitigation Implemented . . . 10
3.3.4 Code Example: SQL Injection Prevention . . . 10
3.4 A04: Insecure Design . . . 11
3.4.1 Description . . . 11
3.4.2 Risk in SYNAPSE Context . . . 11
3.4.3 Mitigation Implemented . . . 11
3.5 A05: Security Misconfiguration . . . 11
3.5.1 Description . . . 11
3.5.2 Risk in SYNAPSE Context . . . 11
3.5.3 Mitigation Implemented . . . 11
3.6 A06: Vulnerable Components . . . 12
3.6.1 Description . . . 12
3.6.2 Risk in SYNAPSE Context . . . 12
3.6.3 Mitigation Implemented . . . 12
3.6.4 Code Example: Dependency Scanning . . . 12
3.7 A07: Identification and Authentication Failures . . . 13
3.7.1 Description . . . 13
3.7.2 Risk in SYNAPSE Context . . . 13
3.7.3 Mitigation Implemented . . . 13
3.8 A08: Software and Data Integrity Failures . . . 13
3.8.1 Description . . . 13
3.8.2 Risk in SYNAPSE Context . . . 13
3.8.3 Mitigation Implemented . . . 14
3.9 A09: Security Logging and Monitoring Failures . . . 14
3.9.1 Description . . . 14
3.9.2 Risk in SYNAPSE Context . . . 14
3.9.3 Mitigation Implemented . . . 14
3.10 A10: Server-Side Request Forgery (SSRF) . . . 14
3.10.1 Description . . . 14
3.10.2 Risk in SYNAPSE Context . . . 15
3.10.3 Mitigation Implemented . . . 15
3.10.4 Code Example: SSRF Prevention . . . 15

4 Authentication and Authorization Security 16
4.1 JWT Implementation . . . 16
4.1.1 Token Structure and Signing . . . 16
4.1.2 JWT Configuration . . . 16
4.2 Token Storage . . . 17
4.2.1 HTTP-Only Cookies (Recommended) . . . 17
4.2.2 Token Storage Implementation . . . 17
4.3 Token Refresh Strategy . . . 18
4.3.1 Refresh Token Rotation . . . 18
4.3.2 Refresh Endpoint . . . 18
4.4 Password Security . . . 18
4.4.1 Password Hashing . . . 18
4.4.2 Password Policy Configuration . . . 19
4.5 Role-Based Access Control (RBAC) . . . 19

4.5.1 Role Hierarchy . . . 19
4.5.2 RBAC Implementation . . . 19
4.6 Django Permission Classes . . . 20
4.7 OAuth2 for Third-Party Integrations . . . 21
4.7.1 Google Drive Integration . . . 21
4.7.2 OAuth2 Configuration . . . 21
4.8 Multi-Factor Authentication (MFA) . . . 21
4.8.1 MFA Support . . . 21
4.9 Account Lockout . . . 22
4.9.1 django-axes Configuration . . . 22
4.10 Session Management . . . 22
4.10.1 Session Configuration . . . 22

5 API Security **23**
5.1 Rate Limiting Strategy . . . 23
5.2 Django REST Framework Throttle Configuration . . . 23
5.3 Redis-Backed Rate Limiting . . . 24
5.4 API Key Management . . . 24
5.4.1 API Key Features . . . 24
5.4.2 API Key Implementation . . . 25
5.5 Input Validation . . . 25
5.5.1 Django REST Framework Serializers . . . 25
5.5.2 Pydantic Validation for FastAPI . . . 26
5.6 Request Size Limits . . . 26
5.7 CORS Configuration . . . 27
5.7.1 CORS Whitelist . . . 27
5.8 GraphQL Security . . . 27
5.8.1 GraphQL Configuration . . . 27
5.8.2 GraphQL Implementation . . . 28
5.9 API Versioning . . . 28

6 Data Security **30**
6.1 Data Classification . . . 30
6.2 Encryption at Rest . . . 30
6.2.1 AES-256 Field-Level Encryption . . . 30
6.2.2 PostgreSQL pgcrypto Extension . . . 30
6.3 Encryption in Transit . . . 31
6.3.1 TLS 1.3 Configuration . . . 31
6.3.2 HSTS (HTTP Strict Transport Security) . . . 31
6.4 PII (Personally Identifiable Information) Handling . . . 32
6.4.1 Minimal Collection Principle . . . 32
6.4.2 PII Data Handling . . . 32
6.5 Database Security . . . 32
6.5.1 Least Privilege Database Users . . . 32
6.5.2 Database Connection Security . . . 33
6.6 Secrets Management . . . 33
6.6.1 AWS Secrets Manager Integration . . . 33
6.6.2 Secrets Retrieval . . . 34
6.7 Data Retention and Deletion Policies . . . 34
6.7.1 Retention Schedule . . . 34
6.7.2 Data Deletion Implementation . . . 35
6.8 Backup Encryption . . . 35
6.9 pgvector/Embedding Data Security . . . 35

7 Django Security Settings **37**
7.1 Complete Security Configuration . . . 37
7.1.1 settings.py Security Checklist . . . 37
7.2 CSRF Protection for API Endpoints . . . 38
7.3 Content Security Policy (CSP) Headers . . . 39
7.4 Django Admin Hardening . . . 40
7.4.1 Admin Security Configuration . . . 40
7.4.2 Admin IP Whitelist Middleware . . . 40

8 Infrastructure Security **42**
8.1 AWS VPC Configuration . . . 42
8.1.1 Network Architecture . . . 42
8.1.2 VPC Diagram (Text Format) . . . 42
8.2 Security Groups . . . 42
8.2.1 ALB Security Group . . . 42
8.2.2 Application Server Security Group . . . 43
8.2.3 RDS Security Group . . . 43
8.2.4 Redis Security Group . . . 43
8.3 EC2 Hardening . . . 43
8.3.1 EC2 Configuration . . . 43
8.3.2 EC2 Launch Template . . . 44
8.4 RDS Security . . . 44
8.4.1 RDS Configuration . . . 44
8.4.2 RDS Creation . . . 45
8.5 S3 Bucket Security . . . 45
8.5.1 Bucket Policy . . . 45
8.5.2 S3 Configuration . . . 46
8.6 Docker Security . . . 47
8.6.1 Dockerfile Security Best Practices . . . 47
8.6.2 Docker Security Scanning . . . 47
8.7 Kubernetes/ECS Task Definition Security . . . 48
8.7.1 ECS Task Definition . . . 48

9 Dependency Security **50**
9.1 Python Dependency Scanning . . . 50
9.1.1 Safety Check . . . 50
9.1.2 Bandit Security Analysis . . . 50
9.1.3 pip-audit Tool . . . 50
9.2 Node.js Dependency Scanning . . . 51
9.2.1 npm audit . . . 51
9.2.2 Snyk Integration . . . 51
9.3 Automated Vulnerability Scanning . . . 51
9.3.1 GitHub Dependabot Configuration . . . 51
9.4 Software Bill of Materials (SBOM) . . . 52
9.4.1 SBOM Generation . . . 52
9.5 Dependency Update Policy . . . 52
9.5.1 CVE Response SLA . . . 52
9.5.2 Dependency Pinning . . . 52

10 Security Monitoring and Incident Response **54**
10.1 Security Logging Requirements . . . 54
10.1.1 Logging Events . . . 54
10.1.2 Structured JSON Logging . . . 54
10.2 Log Aggregation with CloudWatch . . . 55

10.3 SIEM Integration: ELK Stack . . . 55
10.3.1 Logstash Configuration . . . 55
10.4 Intrusion Detection . . . 56
10.4.1 fail2ban Configuration . . . 56
10.4.2 AWS GuardDuty . . . 57
10.5 Alert Thresholds . . . 57
10.5.1 Alert Configuration . . . 57
10.6 Incident Response Plan . . . 57
10.6.1 Incident Response Phases . . . 57
10.6.2 Incident Severity and SLA . . . 58
10.6.3 Incident Response Playbook . . . 58
10.7 Vulnerability Disclosure Policy . . . 58
10.7.1 Security.txt . . . 58
10.7.2 Responsible Disclosure Process . . . 59

11 Penetration Testing Checklist **60**
11.1 Reconnaissance Tests . . . 60
11.2 Authentication Bypass Tests . . . 60
11.3 Authorization Tests (IDOR & Privilege Escalation) . . . 61
11.4 Injection Tests . . . 61
11.5 API Security Tests . . . 61
11.6 Business Logic Tests . . . 62
11.7 Infrastructure Tests . . . 62
11.8 Penetration Testing Tools . . . 62
11.9 Testing Frequency . . . 63

12 GDPR and Privacy Compliance **64**
12.1 Data Subject Rights . . . 64
12.1.1 Right of Access . . . 64
12.1.2 Right to Rectification . . . 64
12.1.3 Right to Erasure (Right to be Forgotten) . . . 64
12.1.4 Right to Data Portability . . . 64
12.1.5 Right to Restrict Processing . . . 64
12.1.6 Right to Object . . . 64
12.2 Privacy by Design . . . 64
12.3 Cookie Consent . . . 65
12.3.1 Cookie Banner Implementation . . . 65
12.3.2 Cookie Categories . . . 65
12.4 Data Processing Agreement (DPA) . . . 65
12.5 Privacy Policy Requirements . . . 66
12.6 Data Breach Notification . . . 66
12.6.1 72-Hour Rule . . . 66
12.6.2 Breach Notification Process . . . 66
12.7 Data Protection Officer (DPO) . . . 66
12.7.1 DPO Responsibilities . . . 66

13 Security Audit Checklist **68**
13.1 Pre-Deployment Security Checklist (50+ Items) . . . 68
13.1.1 Code Security . . . 68
13.1.2 Infrastructure Security . . . 68
13.1.3 Application Security . . . 69
13.1.4 Compliance . . . 69
13.2 Code Review Security Checklist . . . 70
13.3 Quarterly Security Review Tasks . . . 70

13.4 Annual Requirements . . . 71

14 Security Training and Culture **72**
14.1 Developer Security Training . . . 72
14.1.1 Required Topics . . . 72
14.1.2 Training Schedule . . . 72
14.2 Secure Coding Guidelines . . . 73
14.2.1 Golden Rules . . . 73
14.2.2 Code Example: Secure vs. Insecure . . . 73
14.3 Security Champions Program . . . 73
14.3.1 Program Overview . . . 73
14.3.2 Champion Responsibilities . . . 74
14.4 Security Documentation . . . 74

A Additional Resources **75**
A.1 External References . . . 75
A.2 Tools and Resources . . . 75

Chapter 1

Introduction

1.1 Security Philosophy

The SYNAPSE platform follows a comprehensive security approach based on three core principles:

1. **Defense in Depth**: Multiple layers of security controls ensure that if one layer fails, others remain effective. This applies across network, application, data, and operational layers.
2. **Least Privilege**: Every user, process, and component has only the minimum permissions required to perform their function. This principle is applied to database users, AWS IAM roles, API endpoints, and file system access.
3. **Zero Trust Architecture**: We assume no implicit trust. All requests, whether internal or external, must be authenticated and authorized. Network segmentation, encryption, and continuous verification are mandatory.

1.2 Scope of Security Coverage

This document covers security implementations across the entire SYNAPSE stack:

- **Application Layer**: Django 4.2 backend, FastAPI services, Next.js 14 frontend
- **Data Layer**: PostgreSQL 15 database, Redis 7 caching, pgvector embeddings
- **Message Queue**: Celery for asynchronous tasks
- **Infrastructure**: AWS deployment, Docker containerization, VPC configuration
- **Third-party Integrations**: Google Drive OAuth, external APIs
- **Operations**: Monitoring, logging, incident response, vulnerability management

1.3 Security Definitions

1.3.1 OWASP (Open Web Application Security Project)

A non-profit organization focused on improving software security through community-driven projects, tools, and documentation. OWASP publishes the Top 10, a list of the most critical web application security risks.

1.3.2 CVE (Common Vulnerabilities and Exposures)

A system for identifying and cataloging publicly disclosed cybersecurity vulnerabilities. Each CVE receives a unique identifier (e.g., CVE-2023-1234) and severity rating.

1.3.3 CVSS (Common Vulnerability Scoring System)

A standardized method for rating the severity of security vulnerabilities on a scale of 0.0 to 10.0:

- 0.0: None
- 0.1-3.9: Low
- 4.0-6.9: Medium
- 7.0-8.9: High
- 9.0-10.0: Critical

1.3.4 PII (Personally Identifiable Information)

Any data that can be used to identify an individual, including names, email addresses, phone numbers, IP addresses, and account identifiers.

1.3.5 JWT (JSON Web Token)

A stateless token format for securely transmitting claims between parties. Structure: Header.Payload.Signature. Used for authentication and API access control.

1.3.6 RBAC (Role-Based Access Control)

An access control model where permissions are granted based on user roles rather than individual users. SYNAPSE uses three primary roles: Admin, Premium, and User.

1.3.7 STRIDE Threat Model

A methodology for identifying six classes of security threats:

- **Spoofing**: Identity spoofing or impersonation
- **Tampering**: Unauthorized modification of data
- **Repudiation**: Denial of actions or transactions
- **Information Disclosure**: Unauthorized access to sensitive data
- **Denial of Service**: Attacks that prevent system availability
- **Elevation of Privilege**: Unauthorized access to higher-level permissions

1.4 Regulatory Context

SYNAPSE operates under the following compliance frameworks:

- **GDPR (General Data Protection Regulation)**: European privacy regulation requiring data protection, consent, and subject rights
- **CCPA (California Consumer Privacy Act)**: California privacy law similar to GDPR
- **OWASP Top 10**: Industry standard for web application security
- **CWE (Common Weakness Enumeration)**: Categorization of software weaknesses
- **ISO 27001**: International standard for information security management

Chapter 2

Threat Model

2.1 Assets to Protect

2.1.1 User Data

- User accounts, email addresses, profile information
- Authentication credentials and tokens
- Saved searches, preferences, and workspace configurations
- Usage history and activity logs

2.1.2 API Keys and Secrets

- OAuth tokens for Google Drive integration
- Third-party API keys for external services
- Database connection strings
- AWS credentials and access tokens
- JWT signing keys

2.1.3 Scraped Content

- Aggregated technology intelligence data
- Web-scraped content with licensing implications
- Competitive intelligence data
- Patent and research document aggregations

2.1.4 AI Models and Embeddings

- Trained machine learning models
- Vector embeddings stored in pgvector
- Fine-tuning data and training datasets
- Model weights and parameters

2.1.5 Infrastructure

- AWS accounts and resources
- Docker registry and container images
- Database backups and snapshots
- Server configurations and deployment scripts

2.2 Threat Actors

2.2.1 Unauthenticated Users

External attackers with no legitimate access. Threats include reconnaissance, credential stuffing, and exploitation of authentication endpoints.

2.2.2 Authenticated Users

Legitimate users with platform access. Threats include Insecure Direct Object Reference (IDOR), privilege escalation, and data exfiltration.

2.2.3 Admin Users

Internal administrators with elevated privileges. Threats include insider threats, negligent misuse, and account compromise.

2.2.4 External Attackers

Sophisticated threat actors targeting the platform. Threats include APTs (Advanced Persistent Threats), zero-day exploits, and supply chain attacks.

2.2.5 Insiders

Current or former employees with system knowledge. Threats include deliberate sabotage, data theft, and credential misuse.

2.3 Attack Vectors

2.3.1 Web-Based Attacks

- Cross-Site Scripting (XSS): JavaScript injection in frontend
- Cross-Site Request Forgery (CSRF): Unauthorized request forgery
- SQL Injection: Database query manipulation
- Server-Side Template Injection (SSTI): Template engine exploitation

2.3.2 API-Based Attacks

- Broken Authentication: Weak token validation
- Broken Access Control: Privilege escalation via API
- Rate Limit Bypass: Excessive API requests
- Insecure Deserialization: Malicious object serialization

2.3.3 Network-Based Attacks

- Man-in-the-Middle (MITM): TLS interception
- DNS Hijacking: Malicious DNS resolution
- DDoS Attacks: Volumetric attacks on infrastructure
- Network Reconnaissance: Port scanning, vulnerability probing

2.3.4 Supply Chain Attacks

- Dependency Poisoning: Malicious package injection
- Container Image Compromise: Infected Docker images
- Build Pipeline Attacks: Compromised build tools
- Third-party Integration Compromise: Malicious OAuth/API providers

2.3.5 Social Engineering

- Phishing: Credential harvesting via email
- Pretexting: False authority claims
- Baiting: Malicious USB or downloads
- Tailgating: Physical access exploitation

2.4 STRIDE Threat Model with Mitigations

Threat	Description	Mitigation
[CRITICAL] Spoofing	Attacker impersonates legitimate user or service	JWT with RS256 signing, OAuth2 verification, API key management, IP logging
[CRITICAL] Tampering	Unauthorized modification of data in transit or at rest	TLS 1.3 encryption, AES-256 encryption at rest, HMAC signatures, database constraints
[HIGH] Repudiation	User denies performing an action	Comprehensive audit logging, immutable log storage, digital signatures
[CRITICAL] Info Disclosure	Unauthorized access to sensitive data	Encryption, access control, PII minimization, secure logging practices
[CRITICAL] DoS	System becomes unavailable	Rate limiting, auto-scaling, CDN usage, request validation, timeouts
[CRITICAL] Elevation of Privilege	Attacker gains higher permissions	RBAC, least privilege database users, privilege boundary enforcement

2.5 Risk Matrix: Top 15 Threats

Threat	Likelihood	Impact	Risk Level
1. Credential Stuffing	High	High	[CRITICAL]
2. SQL Injection	Medium	Critical	[CRITICAL]
3. Privilege Escalation (IDOR)	Medium	Critical	[CRITICAL]
4. API Rate Limit Bypass	High	High	[CRITICAL]
5. Compromised Dependencies	Medium	Critical	[CRITICAL]
6. XSS via Frontend	High	Medium	[HIGH]
7. CSRF Attacks	Medium	Medium	[HIGH]
8. DDoS on API	Medium	High	[HIGH]
9. JWT Token Compromise	Low	Critical	[HIGH]
10. OAuth Token Theft	Medium	High	[HIGH]
11. Information Disclosure (Logs)	Medium	Medium	[MEDIUM]
12. Insecure Deserialization	Low	High	[MEDIUM]
13. API Key Exposure	Low	High	[MEDIUM]
14. Social Engineering	Low	Medium	[LOW]
15. Physical Security Breach	Very Low	Medium	[LOW]

Chapter 3

OWASP Top 10 (2021) Coverage

3.1 A01: Broken Access Control

3.1.1 Description

Broken Access Control occurs when users can act outside their intended permissions. This includes accessing unauthorized resources, modifying data, changing account settings, or accessing other users' accounts.

3.1.2 Risk in SYNAPSE Context

[CRITICAL] Users could access other users' saved searches, workspace data, or API keys. Admins could be impersonated, allowing unauthorized system modifications.

3.1.3 Mitigation Implemented

- JWT-based authentication with RS256 signing
- RBAC with three roles: Admin, Premium, User
- Django permission classes on all views
- IDOR prevention through user ID validation
- Object-level permissions enforcement
- Admin action logging and audit trails

3.1.4 Code Example: Django Permission Classes

```python
from rest_framework.permissions import BasePermission
from rest_framework.exceptions import PermissionDenied

class IsOwnerOrReadOnly(BasePermission):
    def has_object_permission(self, request, view, obj):
        if request.method in ['GET', 'HEAD', 'OPTIONS']:
            return True
        return obj.owner == request.user

class IsPremiumUser(BasePermission):
    def has_permission(self, request, view):
        return (request.user and request.user.is_authenticated
                and request.user.is_premium)

```

```python
class IsAdmin(BasePermission):
    def has_permission(self, request, view):
        return request.user and request.user.is_staff

# Usage in views
class SearchViewSet(viewsets.ModelViewSet):
    permission_classes = [IsAuthenticated, IsOwnerOrReadOnly]

    def get_queryset(self):
        return Search.objects.filter(user=self.request.user)
```

3.2 A02: Cryptographic Failures

3.2.1 Description

Cryptographic failures occur when sensitive data is exposed due to weak encryption, missing encryption, or insecure key management.

3.2.2 Risk in SYNAPSE Context

[CRITICAL] User emails, API keys, and OAuth tokens could be exposed if stored unencrypted. Database credentials in connection strings could be leaked.

3.2.3 Mitigation Implemented

- AES-256 encryption for sensitive database fields using pgcrypto
- TLS 1.3 minimum for all data in transit
- HSTS (HTTP Strict Transport Security) enabled
- JWT tokens signed with RS256 (asymmetric cryptography)
- Secrets stored in AWS Secrets Manager, never in code
- Secure key rotation policies
- Database SSL/TLS connections enforced

3.2.4 Code Example: Django Encrypted Fields

```python
from django_encrypted_model_fields.fields import EncryptedTextField

class User(models.Model):
    email = models.EmailField()
    encrypted_phone = EncryptedTextField(null=True, blank=True)
    encrypted_oauth_token = EncryptedTextField(null=True)

    class Meta:
        db_table = 'users'

# Encryption happens transparently on save/load
user = User.objects.get(id=1)
phone = user.encrypted_phone  # Automatically decrypted
```

3.3 A03: Injection

3.3.1 Description

Injection attacks occur when untrusted data is sent as commands or queries without proper validation or parameterization.

3.3.2 Risk in SYNAPSE Context

[CRITICAL] SQL injection could expose the entire database. Command injection in Celery tasks could execute arbitrary code. NoSQL injection could manipulate Redis queries.

3.3.3 Mitigation Implemented

- Parameterized queries in Django ORM (prevents SQL injection)
- Pydantic models for FastAPI input validation
- Django REST Framework serializers for validation
- No raw SQL queries without parameterization
- Input length limits enforced at application level
- Whitelist validation for command execution
- Celery task argument validation
- Redis key prefixing and type checking

3.3.4 Code Example: SQL Injection Prevention

```python
# VULNERABLE - NEVER DO THIS
search_term = request.GET.get('q')
results = Search.objects.raw(f"SELECT * FROM searches WHERE title LIKE '%{search_term}%'")

# SECURE - Use ORM parameterization
search_term = request.GET.get('q')
results = Search.objects.filter(title__icontains=search_term)

# Or with Q objects for complex queries
from django.db.models import Q
results = Search.objects.filter(
    Q(title__icontains=search_term) |
    Q(description__icontains=search_term)
)

# Pydantic validation for FastAPI
from pydantic import BaseModel, constr

class SearchQuery(BaseModel):
    query: constr(max_length=256)
    filters: dict = {}

@app.post("/api/search")
async def search(search: SearchQuery):
    results = db.search(search.query)
    return results
```

3.4 A04: Insecure Design

3.4.1 Description

Insecure design refers to missing security controls by design, inadequate threat modeling, or unsafe design patterns.

3.4.2 Risk in SYNAPSE Context

[CRITICAL] Lack of rate limiting design could allow unlimited API abuse. Missing audit logging could hide insider threats. No encryption-by-design could expose sensitive data.

3.4.3 Mitigation Implemented

- Threat modeling performed on all major features
- Security-first design reviews for new features
- Rate limiting baked into API design
- Audit logging on all sensitive operations
- Default-deny approach for permissions
- Secure defaults (SSL enabled, debug mode off)
- Regular security architecture reviews

3.5 A05: Security Misconfiguration

3.5.1 Description

Security misconfiguration occurs when security settings are not properly configured, defaults are used, or unnecessary features are enabled.

3.5.2 Risk in SYNAPSE Context

[CRITICAL] DEBUG mode enabled could leak sensitive information. ALLOWED_HOSTS misconfiguration could enable Host Header Injection. S3 buckets could be publicly accessible.

3.5.3 Mitigation Implemented

- DEBUG=False in production
- SECURE_SSL_REDIRECT=True
- SECURE_HSTS_SECONDS=31536000 (1 year)
- SESSION_COOKIE_SECURE=True
- CSRF_COOKIE_SECURE=True
- SECURE_CONTENT_TYPE_NOSNIFF=True
- SECURE_BROWSER_XSS_FILTER=True
- X_FRAME_OPTIONS='DENY'
- ALLOWED_HOSTS whitelist configured

- S3 bucket public ACL blocking enabled
- Container images scanned before deployment
- Secrets never committed to version control

3.6 A06: Vulnerable Components

3.6.1 Description

Using libraries, frameworks, or components with known vulnerabilities that could be exploited.

3.6.2 Risk in SYNAPSE Context

[CRITICAL] Outdated Django, FastAPI, or Next.js versions could contain publicly known exploits. Vulnerable Python or Node dependencies could be exploited.

3.6.3 Mitigation Implemented

- Automated dependency scanning with Dependabot
- Python: Safety, Bandit, pip-audit tools
- Node.js: npm audit, Snyk scanning
- GitHub Actions CI/CD blocking vulnerable dependencies
- Patch critical CVEs within 24 hours
- Patch high severity CVEs within 7 days
- Software Bill of Materials (SBOM) generated for each release
- Container image scanning with AWS ECR and Trivy
- Regular dependency updates scheduled monthly

3.6.4 Code Example: Dependency Scanning

```
# Python dependency security checking
pip install safety pip-audit bandit

# Check for known vulnerabilities
safety check --json

# Audit specific packages
pip-audit

# Security code analysis
bandit -r . -f json

# GitHub Actions workflow
name: Security Scan
on: [push, pull_request]
jobs:
  security:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
```

```
      - name: Run safety check
        run: |
          pip install safety
          safety check --json > safety-report.json
      - name: Fail on vulnerabilities
        run: safety check
```

3.7 A07: Identification and Authentication Failures

3.7.1 Description

Weak authentication mechanisms that allow attackers to compromise user accounts or bypass authentication entirely.

3.7.2 Risk in SYNAPSE Context

[CRITICAL] Weak password requirements could allow brute force. Unencrypted passwords could be exposed. Token theft could grant unauthorized access.

3.7.3 Mitigation Implemented

- Passwords hashed with bcrypt (work factor 12)
- Minimum 8 character password requirement
- Password complexity: uppercase, lowercase, numbers, special chars
- Account lockout after 5 failed login attempts (django-axes)
- 30-minute lockout duration
- JWT tokens with 15-minute expiration
- Refresh tokens with 7-day expiration
- HTTP-only, secure cookies for token storage
- Token refresh endpoint with rate limiting
- MFA support for premium accounts
- Session timeout after 24 hours of inactivity

3.8 A08: Software and Data Integrity Failures

3.8.1 Description

Failures in software updates, CI/CD pipelines, or data integrity verification that could allow malicious code injection.

3.8.2 Risk in SYNAPSE Context

[CRITICAL] Compromised Docker images could deploy malicious code. Unsigned package updates could introduce vulnerabilities. Tampered database snapshots could corrupt data.

3.8.3 Mitigation Implemented

- Docker image signing and verification
- Container registry scanning (AWS ECR)
- Code review on all commits
- GitHub branch protection requiring reviews
- Signed git commits required
- Automated tests on all pull requests
- SBOM generation and verification
- Database backup verification and integrity checks
- Secure supply chain practices

3.9 A09: Security Logging and Monitoring Failures

3.9.1 Description

Insufficient logging and monitoring that prevents detection of attacks or forensic investigation.

3.9.2 Risk in SYNAPSE Context

[CRITICAL] Without audit logs, insider threats could go undetected. API abuse might not trigger alerts. Breaches could go unnoticed for extended periods.

3.9.3 Mitigation Implemented

- Structured JSON logging with no PII
- All authentication events logged
- All API access logged with timestamps
- All admin actions audited
- Failed login attempts tracked (django-axes)
- Database query logging in development
- CloudWatch integration for log aggregation
- ELK stack (Elasticsearch, Logstash, Kibana) for analysis
- Alert thresholds for suspicious patterns
- Retention policy: 90 days operational, 1 year compliance
- Log immutability enforcement

3.10 A10: Server-Side Request Forgery (SSRF)

3.10.1 Description

Attacks where the server makes unintended requests to internal or external systems on behalf of the attacker.

3.10.2 Risk in SYNAPSE Context

[CRITICAL] An attacker could scan internal AWS metadata service. SSRF could be used to access internal databases or Redis. OAuth token refresh endpoints could be exploited.

3.10.3 Mitigation Implemented

- URL validation whitelist for external requests
- Blocked internal IP ranges (10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16)
- AWS metadata service endpoint blocked (169.254.169.254)
- Request size limits (max 10MB)
- Timeout limits on external requests (5 seconds)
- DNS resolution validation
- VPC configuration with no internet access for databases
- WAF rules to detect SSRF patterns

3.10.4 Code Example: SSRF Prevention

```python
import ipaddress
import requests
from urllib.parse import urlparse

# SSRF-safe request function
def safe_external_request(url, timeout=5):
    # Whitelist of allowed domains
    WHITELIST_DOMAINS = [
        'api.github.com',
        'api.twitter.com',
        'drive.google.com'
    ]

    parsed = urlparse(url)

    # Check domain whitelist
    if parsed.netloc not in WHITELIST_DOMAINS:
        raise ValueError(f"Domain {parsed.netloc} not whitelisted")

    # Block internal IPs
    try:
        ip = socket.gethostbyname(parsed.hostname)
        ip_obj = ipaddress.ip_address(ip)

        # Block private IP ranges
        if ip_obj.is_private or ip_obj.is_loopback:
            raise ValueError(f"Internal IP {ip} blocked")
    except socket.gaierror:
        raise ValueError("DNS resolution failed")

    # Make request with timeout
    response = requests.get(url, timeout=timeout)
    return response
```

Chapter 4

Authentication and Authorization Security

4.1 JWT Implementation

SYNAPSE uses JWT (JSON Web Tokens) for stateless authentication across the platform.

4.1.1 Token Structure and Signing

- **Algorithm**: RS256 (RSA Signature with SHA-256)
- **Asymmetric Signing**: Private key for signing, public key for verification
- **Access Token Expiration**: 15 minutes
- **Refresh Token Expiration**: 7 days
- **Token Claims**: User ID, role, permissions, issued at, expiration

4.1.2 JWT Configuration

```python
# settings.py
import datetime
from pathlib import Path

SIMPLE_JWT = {
    'ACCESS_TOKEN_LIFETIME': datetime.timedelta(minutes=15),
    'REFRESH_TOKEN_LIFETIME': datetime.timedelta(days=7),
    'ROTATE_REFRESH_TOKENS': True,
    'BLACKLIST_AFTER_ROTATION': True,
    'ALGORITHM': 'RS256',
    'SIGNING_KEY': Path('/run/secrets/jwt_private_key').read_text(),
    'VERIFYING_KEY': Path('/run/secrets/jwt_public_key').read_text(),
    'USER_ID_FIELD': 'id',
    'USER_ID_CLAIM': 'user_id',
    'AUTH_TOKEN_CLASSES': ('rest_framework_simplejwt.tokens.AccessToken',),
    'TOKEN_TYPE_CLAIM': 'token_type',
}

REST_FRAMEWORK = {
    'DEFAULT_AUTHENTICATION_CLASSES': (
        'rest_framework_simplejwt.authentication.JWTAuthentication',
```

```
    ),
}
```

4.2 Token Storage

4.2.1 HTTP-Only Cookies (Recommended)

Tokens are stored in HTTP-only, secure, same-site cookies:

- **HTTP-Only**: JavaScript cannot access the token (prevents XSS theft)
- **Secure Flag**: Cookie only sent over HTTPS
- **SameSite=Strict**: Cookie not sent in cross-site requests (CSRF protection)
- **Path=/api/**: Cookie only sent to API endpoints
- **Domain**: Configured for main domain only

4.2.2 Token Storage Implementation

```python
# views.py
from rest_framework_simplejwt.views import TokenObtainPairView

class CookieTokenObtainPairView(TokenObtainPairView):
    def post(self, request, *args, **kwargs):
        response = super().post(request, *args, **kwargs)

        if response.status_code == 200:
            access_token = response.data['access']
            refresh_token = response.data['refresh']

            # Set secure HTTP-only cookies
            response.set_cookie(
                'access_token',
                access_token,
                max_age=15*60,  # 15 minutes
                httponly=True,
                secure=True,
                samesite='Strict',
                path='/api/'
            )

            response.set_cookie(
                'refresh_token',
                refresh_token,
                max_age=7*24*60*60,  # 7 days
                httponly=True,
                secure=True,
                samesite='Strict',
                path='/api/auth/refresh/'
            )

            # Remove tokens from response body
            del response.data['access']
            del response.data['refresh']

        return response
```

4.3 Token Refresh Strategy

4.3.1 Refresh Token Rotation

- Refresh tokens are rotated on each refresh request
- Old refresh tokens are blacklisted after rotation
- Prevents token reuse if a refresh token is stolen
- Suspicious refresh patterns (rapid succession) trigger alerts

4.3.2 Refresh Endpoint

```python
class CookieTokenRefreshView(TokenRefreshView):
    def post(self, request, *args, **kwargs):
        response = super().post(request, *args, **kwargs)

        if response.status_code == 200:
            new_access = response.data['access']
            new_refresh = response.data.get('refresh')

            response.set_cookie(
                'access_token',
                new_access,
                max_age=15*60,
                httponly=True,
                secure=True,
                samesite='Strict'
            )

            if new_refresh:
                response.set_cookie(
                    'refresh_token',
                    new_refresh,
                    max_age=7*24*60*60,
                    httponly=True,
                    secure=True,
                    samesite='Strict'
                )

        return response
```

4.4 Password Security

4.4.1 Password Hashing

- **Algorithm**: bcrypt with work factor 12
- **Minimum Length**: 8 characters
- **Complexity**: Minimum 1 uppercase, 1 lowercase, 1 digit, 1 special char
- **History**: Prevent reuse of last 5 passwords
- **Expiration**: No mandatory expiration (NIST guidance)

4.4.2 Password Policy Configuration

```python
# settings.py
AUTH_PASSWORD_VALIDATORS = [
    {
        'NAME': 'django.contrib.auth.password_validation.UserAttributeSimilarityValidator',
    },
    {
        'NAME': 'django.contrib.auth.password_validation.MinimumLengthValidator',
        'OPTIONS': {'min_length': 8}
    },
    {
        'NAME': 'django.contrib.auth.password_validation.CommonPasswordValidator',
    },
    {
        'NAME': 'django.contrib.auth.password_validation.NumericPasswordValidator',
    },
    {
        'NAME': 'custom_validators.ComplexityValidator',
    },
]

# Custom complexity validator
import re
from django.core.exceptions import ValidationError

class ComplexityValidator:
    def validate(self, password, user=None):
        if not re.search(r'[A-Z]', password):
            raise ValidationError("Password must contain uppercase")
        if not re.search(r'[a-z]', password):
            raise ValidationError("Password must contain lowercase")
        if not re.search(r'[0-9]', password):
            raise ValidationError("Password must contain number")
        if not re.search(r'[!@#$%^&*(),.?":{}|<>]', password):
            raise ValidationError("Password must contain special character")
```

4.5 Role-Based Access Control (RBAC)

4.5.1 Role Hierarchy

4.5.2 RBAC Implementation

```python
# models.py
from django.db import models
from django.contrib.auth.models import AbstractUser

class CustomUser(AbstractUser):
    ROLE_CHOICES = [
        ('admin', 'Administrator'),
        ('premium', 'Premium User'),
        ('user', 'Free User'),
    ]

```

Role	Permissions	Use Case
Admin	All permissions, system configuration, user management	Internal team, system operators
Premium	Full API access, AI features, custom workspaces, API keys, advanced analytics	Paid subscribers
User	Basic API access (100 req/min), limited features	Free tier subscribers
Viewer	Read-only access to shared resources	Invited collaborators (future)

```
    role = models.CharField(max_length=20, choices=ROLE_CHOICES, default='user')
    is_premium = models.BooleanField(default=False)
    mfa_enabled = models.BooleanField(default=False)

    class Meta:
        db_table = 'users'

# permissions.py
from rest_framework.permissions import BasePermission

class IsAdmin(BasePermission):
    def has_permission(self, request, view):
        return request.user and request.user.role == 'admin'

class IsPremium(BasePermission):
    def has_permission(self, request, view):
        return (request.user and
                request.user.role in ['admin', 'premium'])

class IsAuthenticated(BasePermission):
    def has_permission(self, request, view):
        return request.user and request.user.is_authenticated
```

4.6 Django Permission Classes

All views use explicit permission classes:

```
# views.py
from rest_framework import viewsets
from rest_framework.permissions import IsAuthenticated

class SearchViewSet(viewsets.ModelViewSet):
    permission_classes = [IsAuthenticated, IsOwnerOrReadOnly]
    serializer_class = SearchSerializer

    def get_queryset(self):
        return Search.objects.filter(user=self.request.user)

    def perform_create(self, serializer):
        serializer.save(user=self.request.user)

```

```python
class PremiumFeatureViewSet(viewsets.ViewSet):
    permission_classes = [IsPremium]

    def create(self, request):
        if not request.user.is_premium:
            return Response({'error': 'Premium required'},
                            status=403)
        # Process request
```

4.7 OAuth2 for Third-Party Integrations

4.7.1 Google Drive Integration

- OAuth2 code flow with PKCE
- Scopes limited to read-only Drive access
- Tokens encrypted and stored securely
- Refresh tokens rotated on use
- User can revoke access at any time

4.7.2 OAuth2 Configuration

```python
# OAuth2 settings
OAUTH2_PROVIDER = {
    'SCOPES': {
        'read': 'Read access to protected resources',
        'write': 'Write access to protected resources',
    },
    'ACCESS_TOKEN_EXPIRE_SECONDS': 900,
}

# Google Drive OAuth
GOOGLE_OAUTH_SCOPES = [
    'https://www.googleapis.com/auth/drive.readonly',
]

GOOGLE_CLIENT_ID = 'from-aws-secrets-manager'
GOOGLE_CLIENT_SECRET = 'from-aws-secrets-manager'
```

4.8 Multi-Factor Authentication (MFA)

4.8.1 MFA Support

- Required for admin accounts
- Optional for premium users
- TOTP (Time-based One-Time Password) with authenticator apps
- Backup codes provided during setup
- MFA status checked at login

4.9 Account Lockout

4.9.1 django-axes Configuration

```
# settings.py
AXES_FAILURE_LIMIT = 5
AXES_COOLOFF_DURATION = datetime.timedelta(minutes=30)
AXES_LOCK_OUT_BY_COMBINATION_USER_AND_IP = True
AXES_USE_USER_AGENT = True
AXES_VERBOSE = True

# Custom lockout logic
from axes.decorators import axes_dispatch_decorator

@axes_dispatch_decorator
def login_view(request):
    if request.method == 'POST':
        username = request.POST.get('username')
        password = request.POST.get('password')

        user = authenticate(request, username=username, password=password)
        if user is not None:
            login(request, user)
            return redirect('home')
        # Failed attempt logged by axes

    return render(request, 'login.html')
```

4.10 Session Management

4.10.1 Session Configuration

```
# settings.py
SESSION_COOKIE_AGE = 24 * 60 * 60  # 24 hours
SESSION_COOKIE_SECURE = True
SESSION_COOKIE_HTTPONLY = True
SESSION_COOKIE_SAMESITE = 'Strict'
SESSION_EXPIRE_AT_BROWSER_CLOSE = False
SESSION_COOKIE_DOMAIN = '.synapse.local'
SESSION_COOKIE_PATH = '/api/'
```

Chapter 5

API Security

5.1 Rate Limiting Strategy

API rate limiting prevents abuse and controls costs:

User Type	Limit	Window
Unauthenticated	20 requests	1 minute
Free User	100 requests	1 minute
Premium User	1000 requests	1 minute
AI Endpoints	10 requests	1 minute
Admin	Unlimited	N/A

5.2 Django REST Framework Throttle Configuration

```python
# settings.py
REST_FRAMEWORK = {
    'DEFAULT_THROTTLE_CLASSES': [
        'rest_framework.throttling.AnonRateThrottle',
        'rest_framework.throttling.UserRateThrottle',
    ],
    'DEFAULT_THROTTLE_RATES': {
        'anon': '20/min',
        'user': '100/min',
        'premium': '1000/min',
        'ai': '10/min',
    }
}

# Custom throttle classes
from rest_framework.throttling import BaseThrottle

class PremiumUserThrottle(BaseThrottle):
    scope = 'premium'

    def throttle_success(self, request):
        if request.user and request.user.is_premium:
            self.scope = 'premium'
            self.rate = self.get_rate()
            self.num_requests, self.duration = self.parse_rate(self.rate)
            return self.allow_request(request, view)
```

```
        return True

class AIEndpointThrottle(BaseThrottle):
    scope = 'ai'
    THROTTLE_RATES = {'ai': '10/min'}

    def allow_request(self, request, view):
        # Custom logic for AI endpoints
        pass
```

5.3 Redis-Backed Rate Limiting

```
# Use django-ratelimit with Redis backend
import redis
from django_ratelimit.decorators import ratelimit

RATELIMIT_ENABLE = True
RATELIMIT_USE_CACHE = 'redis'

cache_conn = redis.Redis(
    host='redis.internal',
    port=6379,
    db=0,
    ssl=True,
    decode_responses=True
)

# Per-user rate limiting
@ratelimit(key='user', rate='100/m', method=['POST', 'PUT', 'DELETE'])
def api_endpoint(request):
    pass

# Per-IP rate limiting
@ratelimit(key='ip', rate='20/m')
def public_endpoint(request):
    pass
```

5.4 API Key Management

5.4.1 API Key Features

- Generated securely with cryptographically random strings
- Stored hashed (not plaintext) in database
- Can be revoked at any time
- Rate limit per key
- Labeled for identification
- Last used timestamp tracked

5.4.2 API Key Implementation

```python
# models.py
import secrets
from django.db import models

class APIKey(models.Model):
    user = models.ForeignKey(User, on_delete=models.CASCADE)
    key_hash = models.CharField(max_length=255, unique=True)
    name = models.CharField(max_length=100)
    is_active = models.BooleanField(default=True)
    last_used = models.DateTimeField(null=True)
    created_at = models.DateTimeField(auto_now_add=True)

    def save(self, *args, **kwargs):
        if not self.key_hash:
            raw_key = secrets.token_urlsafe(32)
            self.key_hash = hash_api_key(raw_key)
        super().save(*args, **kwargs)

# Authentication
from rest_framework.authentication import TokenAuthentication

class APIKeyAuthentication(TokenAuthentication):
    keyword = 'Bearer'

    def authenticate_credentials(self, key):
        key_hash = hash_api_key(key)
        try:
            api_key = APIKey.objects.get(
                key_hash=key_hash,
                is_active=True
            )
            api_key.last_used = now()
            api_key.save(update_fields=['last_used'])
            return (api_key.user, key)
        except APIKey.DoesNotExist:
            raise exceptions.AuthenticationFailed('Invalid API key')
```

5.5 Input Validation

5.5.1 Django REST Framework Serializers

```python
from rest_framework import serializers

class SearchSerializer(serializers.ModelSerializer):
    query = serializers.CharField(
        max_length=256,
        min_length=1,
        required=True
    )
    filters = serializers.JSONField(required=False)
    limit = serializers.IntegerField(
        min_value=1,
        max_value=100,
        default=10
```

```
    )

    class Meta:
        model = Search
        fields = ['query', 'filters', 'limit']

    def validate_query(self, value):
        if len(value.strip()) == 0:
            raise serializers.ValidationError("Query cannot be empty")
        return value

    def validate_filters(self, value):
        # Whitelist allowed filter keys
        allowed_keys = {'category', 'date_from', 'date_to'}
        if not set(value.keys()).issubset(allowed_keys):
            raise serializers.ValidationError("Invalid filter keys")
        return value
```

5.5.2 Pydantic Validation for FastAPI

```
from pydantic import BaseModel, Field, constr
from typing import Optional

class SearchRequest(BaseModel):
    query: constr(max_length=256, min_length=1)
    filters: Optional[dict] = None
    limit: int = Field(default=10, ge=1, le=100)

    class Config:
        json_schema_extra = {
            "example": {
                "query": "machine learning",
                "filters": {"category": "AI"},
                "limit": 20
            }
        }

@app.post("/search")
async def search(req: SearchRequest):
    # Pydantic validates input automatically
    results = search_service.find(req.query, req.filters)
    return {"results": results}
```

5.6 Request Size Limits

```
# settings.py
FILE_UPLOAD_MAX_MEMORY_SIZE = 10 * 1024 * 1024  # 10MB
DATA_UPLOAD_MAX_MEMORY_SIZE = 10 * 1024 * 1024  # 10MB

# FastAPI
from fastapi import FastAPI, UploadFile

app = FastAPI()

@app.post("/upload")
```

```python
async def upload(file: UploadFile):
    MAX_SIZE = 10 * 1024 * 1024
    size = 0

    content = await file.read(MAX_SIZE + 1)
    if len(content) > MAX_SIZE:
        raise HTTPException(
            status_code=413,
            detail="File too large"
        )

    # Process file
    return {"status": "success"}
```

5.7 CORS Configuration

5.7.1 CORS Whitelist

```python
# settings.py
CORS_ALLOWED_ORIGINS = [
    "https://synapse.io",
    "https://www.synapse.io",
    "https://app.synapse.io",
]

CORS_ALLOW_CREDENTIALS = True
CORS_ALLOW_HEADERS = [
    'accept',
    'accept-encoding',
    'authorization',
    'content-type',
    'dnt',
    'origin',
    'user-agent',
    'x-csrftoken',
    'x-requested-with',
]

CORS_ALLOW_METHODS = [
    'DELETE',
    'GET',
    'OPTIONS',
    'PATCH',
    'POST',
    'PUT',
]

# NO wildcard - explicit whitelist only
CORS_ALLOW_ALL_ORIGINS = False
```

5.8 GraphQL Security

5.8.1 GraphQL Configuration

- Introspection disabled in production

- Query depth limiting (max depth 10)
- Query complexity analysis
- Rate limiting per query type
- Authentication required

5.8.2 GraphQL Implementation

```python
from django.urls import path
from graphene_django.views import GraphQLView
from graphql_core import graphql_sync

class SecureGraphQLView(GraphQLView):
    def parse_body(self, request):
        # Limit query size
        body = super().parse_body(request)
        if len(str(body)) > 10000:
            raise ValueError("Query too large")
        return body

    def get_root_value(self, request, *args, **kwargs):
        # Disable introspection in production
        if settings.DEBUG:
            return super().get_root_value(request, *args, **kwargs)

        # Block introspection queries
        query = self.get_query(request)
        if '__schema' in query or '__type' in query:
            raise ValueError("Introspection disabled")

        return super().get_root_value(request, *args, **kwargs)

urlpatterns = [
    path('graphql/', SecureGraphQLView.as_view(
        graphiql=settings.DEBUG,
        schema=schema
    )),
]
```

5.9 API Versioning

```python
# settings.py
REST_FRAMEWORK = {
    'DEFAULT_VERSIONING_CLASS': 'rest_framework.versioning.URLPathVersioning',
}

# urls.py
from django.urls import path, include
from rest_framework.routers import DefaultRouter

router_v1 = DefaultRouter()
router_v1.register(r'searches', SearchViewSet, basename='search')

urlpatterns = [
```

```
    path('api/v1/', include(router_v1.urls)),
    path('api/v2/', include(router_v2.urls)),
]

# View versioning
class SearchViewSet(viewsets.ModelViewSet):
    def get_serializer_class(self):
        if self.request.version == '1':
            return SearchSerializerV1
        return SearchSerializerV2
```

Chapter 6

Data Security

6.1 Data Classification

SYNAPSE classifies data into four categories:

Level	Examples	Protection
Public	Blog posts, marketing content	No encryption required
Internal	Design docs, architectural diagrams	Encrypted at rest, access controlled
Confidential	User data, API keys, credentials	Strong encryption, minimal access
Restricted	Payment info, healthcare data	Highest encryption, extreme access control

6.2 Encryption at Rest

6.2.1 AES-256 Field-Level Encryption

- User emails encrypted with AES-256
- OAuth tokens encrypted with AES-256
- API keys encrypted before storage
- Encryption keys rotated annually
- Key derivation using PBKDF2

6.2.2 PostgreSQL pgcrypto Extension

```python
# models.py
from django.db import models
from django.contrib.postgres.fields import JSONField

class User(models.Model):
    # Encrypted fields
    email = models.CharField(max_length=255)
    encrypted_phone = models.TextField()
    encrypted_oauth_token = models.TextField(null=True)
```

```python

    class Meta:
        db_table = 'users'

# Database trigger for encryption
# CREATE TRIGGER encrypt_sensitive_data
# BEFORE INSERT ON users
# FOR EACH ROW
# EXECUTE FUNCTION encrypt_sensitive_columns();

# Migration with encryption
class Migration(migrations.Migration):
    operations = [
        migrations.RunSQL(
            sql="""
            UPDATE users
            SET encrypted_phone = pgp_sym_encrypt(
                phone,
                current_setting('app.encryption_key')
            );
            """
        ),
    ]
```

6.3 Encryption in Transit

6.3.1 TLS 1.3 Configuration

- **Minimum Version**: TLS 1.3 only (TLS 1.2 with strong cipher suites acceptable)
- **Cipher Suites**: Only AEAD ciphers (AES-GCM, ChaCha20-Poly1305)
- **Certificate**: Wildcard or SAN certificate from trusted CA
- **Certificate Pinning**: Public key pins for API clients (optional)
- **OCSP Stapling**: Enabled for certificate validation

6.3.2 HSTS (HTTP Strict Transport Security)

```python
# settings.py
SECURE_SSL_REDIRECT = True
SECURE_HSTS_SECONDS = 31536000  # 1 year
SECURE_HSTS_INCLUDE_SUBDOMAINS = True
SECURE_HSTS_PRELOAD = True  # Include in HSTS preload list

# Nginx configuration
server {
    listen 443 ssl http2;
    server_name synapse.io;

    ssl_protocols TLSv1.3;
    ssl_ciphers HIGH:!aNULL:!MD5;
    ssl_prefer_server_ciphers on;

    add_header Strict-Transport-Security
        "max-age=31536000; includeSubDomains; preload"
```

```
        always;
}
```

6.4 PII (Personally Identifiable Information) Handling

6.4.1 Minimal Collection Principle

- Collect only email and name (no phone, SSN, or DOB without consent)
- Purpose limitation: Data used only for stated purposes
- Storage minimization: Delete old data according to retention policy
- User consent required before collection

6.4.2 PII Data Handling

```python
# models.py
class UserProfile(models.Model):
    user = models.OneToOneField(User, on_delete=models.CASCADE)
    # Only essential PII
    full_name = models.CharField(max_length=255)
    email = models.EmailField()
    # NOT collected
    # phone_number, ssn, date_of_birth, passport_number

    # Consent tracking
    marketing_consent = models.BooleanField(default=False)
    privacy_policy_accepted = models.DateTimeField()

    class Meta:
        db_table = 'user_profiles'

# PII access logging
from django.db.models.signals import post_access

def log_pii_access(sender, instance, **kwargs):
    AuditLog.objects.create(
        action='PII_ACCESS',
        user=current_user,
        object_type='UserProfile',
        object_id=instance.id,
        timestamp=now()
    )

post_access.connect(log_pii_access, sender=UserProfile)
```

6.5 Database Security

6.5.1 Least Privilege Database Users

```sql
-- Create read-only user
CREATE USER synapse_read ENCRYPTED PASSWORD 'strong_password';
GRANT CONNECT ON DATABASE synapse TO synapse_read;
```

```sql
GRANT USAGE ON SCHEMA public TO synapse_read;
GRANT SELECT ON ALL TABLES IN SCHEMA public TO synapse_read;

-- Create application user (write)
CREATE USER synapse_app ENCRYPTED PASSWORD 'strong_password';
GRANT CONNECT ON DATABASE synapse TO synapse_app;
GRANT USAGE ON SCHEMA public TO synapse_app;
GRANT SELECT, INSERT, UPDATE, DELETE ON ALL TABLES
    IN SCHEMA public TO synapse_app;

-- Create admin user
CREATE USER synapse_admin ENCRYPTED PASSWORD 'strong_password';
GRANT ALL PRIVILEGES ON DATABASE synapse TO synapse_admin;

-- Deny public access
REVOKE ALL ON DATABASE synapse FROM PUBLIC;

-- Require SSL connections
-- In postgresql.conf: ssl = on
-- In pg_hba.conf: hostssl all all 0.0.0.0/0 cert
```

6.5.2 Database Connection Security

```python
# settings.py
DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.postgresql',
        'NAME': 'synapse',
        'USER': 'synapse_app',
        'PASSWORD': get_secret('DB_PASSWORD'),
        'HOST': 'rds.internal.aws.synapse.local',
        'PORT': '5432',
        'OPTIONS': {
            'sslmode': 'require',
            'sslcert': '/run/secrets/db_cert',
            'sslkey': '/run/secrets/db_key',
            'sslrootcert': '/run/secrets/db_root_cert',
        },
        'CONN_MAX_AGE': 600,
    }
}
```

6.6 Secrets Management

6.6.1 AWS Secrets Manager Integration

- All secrets stored in AWS Secrets Manager
- Never committed to version control
- Secrets Manager audit logging enabled
- Key rotation every 90 days
- Least privilege IAM policies for access

6.6.2 Secrets Retrieval

```python
# secrets.py
import json
import boto3
from functools import lru_cache

class SecretsManager:
    def __init__(self):
        self.client = boto3.client('secretsmanager')

    @lru_cache(maxsize=128)
    def get_secret(self, secret_name):
        try:
            response = self.client.get_secret_value(
                SecretId=secret_name
            )
            return json.loads(response['SecretString'])
        except Exception as e:
            raise ValueError(f"Failed to get secret: {secret_name}")

    def get_database_password(self):
        secrets = self.get_secret('synapse/db/password')
        return secrets['password']

    def get_jwt_keys(self):
        secrets = self.get_secret('synapse/jwt/keys')
        return secrets['private_key'], secrets['public_key']

    def get_api_keys(self):
        return self.get_secret('synapse/api/keys')

secrets = SecretsManager()

# In settings.py
DATABASES = {
    'default': {
        'PASSWORD': secrets.get_database_password(),
    }
}

JWT_PRIVATE_KEY = secrets.get_jwt_keys()[0]
JWT_PUBLIC_KEY = secrets.get_jwt_keys()[1]
```

6.7 Data Retention and Deletion Policies

6.7.1 Retention Schedule

- **Operational Logs**: 90 days
- **Audit Logs**: 1 year
- **User Data**: Until account deletion + 30 days
- **Deleted User Data**: Cryptographic shredding, 7 days
- **Backups**: 30 days for incremental, 90 days for full
- **API Logs**: 30 days

6.7.2 Data Deletion Implementation

```python
# models.py
from django.utils import timezone
from datetime import timedelta

class User(models.Model):
    email = models.EmailField()
    deleted_at = models.DateTimeField(null=True)

    def soft_delete(self):
        self.deleted_at = timezone.now()
        self.save()

    def can_be_permanently_deleted(self):
        if not self.deleted_at:
            return False
        return timezone.now() >= self.deleted_at + timedelta(days=30)

# Celery task for permanent deletion
from celery import shared_task

@shared_task
def delete_old_users():
    users = User.objects.filter(
        deleted_at__lt=timezone.now() - timedelta(days=30)
    )

    for user in users:
        # Cryptographic shredding
        user_data = user.__dict__
        for field in user_data:
            if hasattr(user, field):
                setattr(user, field, None)
        user.delete()
```

6.8 Backup Encryption

- All RDS snapshots encrypted with AWS KMS
- Cross-region backup replication encrypted
- Backup integrity verified with checksums
- Backup restoration tested monthly
- Backup access restricted to backup administrators

6.9 pgvector/Embedding Data Security

```python
# models.py
from pgvector.django import VectorField

class DocumentEmbedding(models.Model):
    document = models.ForeignKey(Document, on_delete=models.CASCADE)
    embedding = VectorField(dimensions=1536)
```

```python
    created_at = models.DateTimeField(auto_now_add=True)

    class Meta:
        db_table = 'document_embeddings'
        indexes = [
            models.Index(fields=['document']),
        ]

# Embeddings are user-specific
# Never share embeddings across users
# Embeddings treated as Confidential data (encrypted at rest)

# Vector search with access control
def search_embeddings(user, query_embedding):
    return DocumentEmbedding.objects.filter(
        document__owner=user
    ).order_by(Distance('embedding', query_embedding))[:10]
```

Chapter 7

Django Security Settings

7.1 Complete Security Configuration

7.1.1 settings.py Security Checklist

```
# settings.py - Production Security Configuration

# DEBUG DISABLED
DEBUG = False
ALLOWED_HOSTS = ['synapse.io', 'www.synapse.io', 'api.synapse.io']

# SSL/TLS Configuration
SECURE_SSL_REDIRECT = True
SECURE_PROXY_SSL_HEADER = ('HTTP_X_FORWARDED_PROTO', 'https')
SESSION_COOKIE_SECURE = True
CSRF_COOKIE_SECURE = True

# HSTS Configuration
SECURE_HSTS_SECONDS = 31536000
SECURE_HSTS_INCLUDE_SUBDOMAINS = True
SECURE_HSTS_PRELOAD = True

# Content Security
SECURE_CONTENT_TYPE_NOSNIFF = True
SECURE_BROWSER_XSS_FILTER = True
X_FRAME_OPTIONS = 'DENY'
SECURE_REFERRER_POLICY = 'strict-origin-when-cross-origin'

# Cookie Configuration
SESSION_COOKIE_HTTPONLY = True
SESSION_COOKIE_SAMESITE = 'Strict'
CSRF_COOKIE_HTTPONLY = True
CSRF_COOKIE_SAMESITE = 'Strict'

# Authentication
AUTH_PASSWORD_VALIDATORS = [
    {'NAME':
        'django.contrib.auth.password_validation.UserAttributeSimilarityValidator'},
    {'NAME': 'django.contrib.auth.password_validation.MinimumLengthValidator',
     'OPTIONS': {'min_length': 8}},
    {'NAME': 'django.contrib.auth.password_validation.CommonPasswordValidator'},
    {'NAME': 'django.contrib.auth.password_validation.NumericPasswordValidator'},
]
```

```python

# Logging Configuration
LOGGING = {
    'version': 1,
    'disable_existing_loggers': False,
    'formatters': {
        'json': {
            '()': 'pythonjsonlogger.jsonlogger.JsonFormatter',
            'format': '%(asctime)s %(name)s %(levelname)s %(message)s'
        }
    },
    'handlers': {
        'file': {
            'level': 'INFO',
            'class': 'logging.handlers.RotatingFileHandler',
            'filename': '/var/log/synapse/django.log',
            'maxBytes': 1024 * 1024 * 100,  # 100MB
            'backupCount': 10,
            'formatter': 'json',
        },
    },
    'root': {
        'handlers': ['file'],
        'level': 'INFO',
    },
}

# CSRF Protection
CSRF_TRUSTED_ORIGINS = ['https://synapse.io']
CSRF_FAILURE_VIEW = 'api.views.csrf_failure'

# Security Middleware
MIDDLEWARE = [
    'django.middleware.security.SecurityMiddleware',
    'django.middleware.csrf.CsrfViewMiddleware',
    'django.middleware.clickjacking.XFrameOptionsMiddleware',
    'corsheaders.middleware.CorsMiddleware',
    'django.middleware.common.CommonMiddleware',
]

# Allowed Hosts
ALLOWED_HOSTS = ['*.synapse.io', 'synapse.io']

# Secret Key
SECRET_KEY = get_secret('DJANGO_SECRET_KEY')

# Database
DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.postgresql',
        'OPTIONS': {'sslmode': 'require'},
    }
}
```

7.2 CSRF Protection for API Endpoints

```python
# views.py
```

```python
from django.middleware.csrf import get_token
from django.views.decorators.csrf import csrf_protect, ensure_csrf_cookie

@ensure_csrf_cookie
def get_csrf_token(request):
    token = get_token(request)
    return JsonResponse({'csrfToken': token})

@csrf_protect
def api_endpoint(request):
    if request.method == 'POST':
        # CSRF token already validated by decorator
        return handle_post(request)
    return JsonResponse({'status': 'ok'})

# For SPA with fetch API
# Include CSRF token in header
// frontend.js
fetch('/api/endpoint', {
    method: 'POST',
    headers: {
        'X-CSRFToken': document.querySelector('meta[name="csrf-token"]').content,
        'Content-Type': 'application/json',
    },
    body: JSON.stringify(data),
    credentials: 'same-origin',
})
```

7.3 Content Security Policy (CSP) Headers

```python
# middleware.py
class SecurityHeadersMiddleware:
    def __init__(self, get_response):
        self.get_response = get_response

    def __call__(self, request):
        response = self.get_response(request)

        # Content Security Policy
        response['Content-Security-Policy'] = (
            "default-src␣'self';␣"
            "script-src␣'self'␣cdn.jsdelivr.net;␣"
            "style-src␣'self'␣'unsafe-inline';␣"
            "img-src␣'self'␣data:␣https:;␣"
            "font-src␣'self';␣"
            "connect-src␣'self';␣"
            "frame-ancestors␣'none';␣"
            "base-uri␣'self';␣"
            "form-action␣'self'"
        )

        # Additional security headers
        response['X-Content-Type-Options'] = 'nosniff'
        response['X-Frame-Options'] = 'DENY'
        response['X-XSS-Protection'] = '1;␣mode=block'
        response['Referrer-Policy'] = 'strict-origin-when-cross-origin'
```

```
        response['Permissions-Policy'] = 'geolocation=(), microphone=(), camera=()'

        return response
```

7.4 Django Admin Hardening

7.4.1 Admin Security Configuration

```
# settings.py
# Change admin URL
ADMIN_URL = get_secret('ADMIN_URL_PATH')  # Not /admin/

# Admin authentication
from django.contrib.admin import AdminSite

class SecureAdminSite(AdminSite):
    site_header = 'SYNAPSE Admin Panel'

    def has_permission(self, request):
        # Require authentication + 2FA
        if not request.user.is_authenticated:
            return False
        if not request.user.is_staff:
            return False
        # Check 2FA enabled
        if not request.user.is_totp_verified:
            return False
        return True

    def get_app_list(self, request):
        # Log admin access
        AuditLog.objects.create(
            action='ADMIN_ACCESS',
            user=request.user,
            ip_address=get_client_ip(request),
        )
        return super().get_app_list(request)

# Use secure admin site
admin.site.__class__ = SecureAdminSite

# IP whitelist for admin
ADMIN_IP_WHITELIST = ['10.0.1.0/24', '203.0.113.0/24']

# Admin rate limiting
from django_ratelimit.decorators import ratelimit

@ratelimit(key='user', rate='50/h', method='POST')
def admin_login(request):
    pass
```

7.4.2 Admin IP Whitelist Middleware

```python
# middleware.py
import ipaddress

class AdminIPWhitelistMiddleware:
    ADMIN_IP_WHITELIST = [
        ipaddress.ip_network('10.0.1.0/24'),
        ipaddress.ip_network('203.0.113.0/24'),
    ]

    def __init__(self, get_response):
        self.get_response = get_response

    def __call__(self, request):
        if request.path.startswith('/secure-admin-path/'):
            client_ip = self.get_client_ip(request)
            ip_obj = ipaddress.ip_address(client_ip)

            # Check if IP is whitelisted
            whitelisted = any(
                ip_obj in network
                for network in self.ADMIN_IP_WHITELIST
            )

            if not whitelisted:
                return JsonResponse(
                    {'error': 'Access denied'},
                    status=403
                )

        return self.get_response(request)

    def get_client_ip(self, request):
        x_forwarded_for = request.META.get('HTTP_X_FORWARDED_FOR')
        if x_forwarded_for:
            ip = x_forwarded_for.split(',')[0]
        else:
            ip = request.META.get('REMOTE_ADDR')
        return ip
```

Chapter 8

Infrastructure Security

8.1 AWS VPC Configuration

8.1.1 Network Architecture

- **Public Subnets**: ALB (Application Load Balancer) only
- **Private Subnets**: Django/FastAPI application servers
- **Database Subnets**: PostgreSQL RDS (no internet access)
- **Cache Subnets**: Redis ElastiCache (no internet access)
- **NAT Gateway**: For outbound internet access from private subnets
- **VPC Flow Logs**: Enabled for traffic analysis

8.1.2 VPC Diagram (Text Format)

```
Internet
   |
   v
[IGW - Internet Gateway]
   |
   +---> [Public Subnet - ALB]
            |
            | (Port 443 only)
            v
[Private Subnet - App Servers] <- EC2 instances
            |
            | (Database queries)
            v
[Private Subnet - RDS PostgreSQL]

[Private Subnet - ElastiCache Redis]
```

8.2 Security Groups

8.2.1 ALB Security Group

```
# Inbound rules
- Protocol: TCP, Port: 80, Source: 0.0.0.0/0 (redirect to 443)
- Protocol: TCP, Port: 443, Source: 0.0.0.0/0

# Outbound rules
- Protocol: TCP, Port: 8000, Destination: app-sg
```

8.2.2 Application Server Security Group

```
# Inbound rules
- Protocol: TCP, Port: 8000, Source: alb-sg
- Protocol: TCP, Port: 22, Source: bastion-sg (SSH via bastion)

# Outbound rules
- Protocol: TCP, Port: 5432, Destination: rds-sg (PostgreSQL)
- Protocol: TCP, Port: 6379, Destination: redis-sg (Redis)
- Protocol: TCP, Port: 443, Destination: 0.0.0.0/0 (HTTPS to internet)
```

8.2.3 RDS Security Group

```
# Inbound rules
- Protocol: TCP, Port: 5432, Source: app-sg

# Outbound rules
- None (deny all outbound)
```

8.2.4 Redis Security Group

```
# Inbound rules
- Protocol: TCP, Port: 6379, Source: app-sg

# Outbound rules
- None (deny all outbound)
```

8.3 EC2 Hardening

8.3.1 EC2 Configuration

- No SSH access from internet (use AWS Systems Manager Session Manager)
- IAM instance profile with minimal permissions
- EBS volumes encrypted with AWS KMS
- VPC flow logs enabled
- CloudWatch agent for monitoring
- Root volume snapshot encryption
- Automatic security patching enabled

8.3.2 EC2 Launch Template

```
aws ec2 create-launch-template \
  --launch-template-name synapse-app-template \
  --launch-template-data '{
    "ImageId": "ami-hardened-app-image",
    "InstanceType": "t3.large",
    "KeyName": "removed-no-ssh",
    "IamInstanceProfile": {"Arn": "arn:aws:iam::ACCOUNT:instance-profile/synapse-app"},
    "SecurityGroupIds": ["sg-app-servers"],
    "BlockDeviceMappings": [
      {
        "DeviceName": "/dev/xvda",
        "Ebs": {
          "VolumeSize": 50,
          "VolumeType": "gp3",
          "Encrypted": true,
          "KmsKeyId": "arn:aws:kms:region:account:key/id",
          "DeleteOnTermination": true
        }
      }
    ],
    "MetadataOptions": {
      "HttpTokens": "required",
      "HttpPutResponseHopLimit": 1
    },
    "Monitoring": {"Enabled": true},
    "TagSpecifications": [
      {
        "ResourceType": "instance",
        "Tags": [
          {"Key": "Name", "Value": "synapse-app-server"},
          {"Key": "Environment", "Value": "production"},
          {"Key": "Compliance", "Value": "required"}
        ]
      }
    ]
  }'
```

8.4 RDS Security

8.4.1 RDS Configuration

- **Engine**: PostgreSQL 15.x
- **Encryption at Rest**: AES-256 with AWS KMS
- **Encryption in Transit**: SSL/TLS required
- **Public Access**: Disabled
- **Backup Retention**: 30 days
- **Multi-AZ**: Enabled for failover
- **Enhanced Monitoring**: CloudWatch detailed metrics
- **Deletion Protection**: Enabled

8.4.2 RDS Creation

```
aws rds create-db-instance \
  --db-instance-identifier synapse-prod \
  --db-instance-class db.r5.xlarge \
  --engine postgres \
  --engine-version 15.3 \
  --master-username postgres \
  --master-user-password GENERATED \
  --allocated-storage 100 \
  --storage-type gp3 \
  --storage-encrypted \
  --kms-key-id arn:aws:kms:region:account:key/id \
  --db-subnet-group-name synapse-db-subnet \
  --vpc-security-group-ids sg-rds-postgres \
  --publicly-accessible false \
  --backup-retention-period 30 \
  --multi-az \
  --enable-cloudwatch-logs-exports postgresql \
  --enable-iam-database-authentication \
  --deletion-protection \
  --copy-tags-to-snapshot \
  --enable-enhanced-monitoring \
  --monitoring-role-arn arn:aws:iam::ACCOUNT:role/rds-monitoring
```

8.5 S3 Bucket Security

8.5.1 Bucket Policy

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "DenyUnencryptedObjectUploads",
      "Effect": "Deny",
      "Principal": "*",
      "Action": "s3:PutObject",
      "Resource": "arn:aws:s3:::synapse-data/*",
      "Condition": {
        "StringNotEquals": {
          "s3:x-amz-server-side-encryption": "AES256"
        }
      }
    },
    {
      "Sid": "DenyPublicAccess",
      "Effect": "Deny",
      "Principal": "*",
      "Action": ["s3:GetObject", "s3:PutObject"],
      "Resource": "arn:aws:s3:::synapse-data/*"
    },
    {
      "Sid": "AllowApplicationAccess",
      "Effect": "Allow",
      "Principal": {
        "AWS": "arn:aws:iam::ACCOUNT:role/synapse-app"
```

```
      },
      "Action": ["s3:GetObject", "s3:PutObject"],
      "Resource": "arn:aws:s3:::synapse-data/*",
      "Condition": {
        "StringEquals": {
          "s3:x-amz-server-side-encryption": "AES256"
        }
      }
    }
  ]
}
```

8.5.2 S3 Configuration

```
# Block all public access
aws s3api put-public-access-block \
  --bucket synapse-data \
  --public-access-block-configuration \
  "BlockPublicAcls=true,IgnorePublicAcls=true,BlockPublicPolicy=true,RestrictPublicBuckets=true"

# Enable versioning
aws s3api put-bucket-versioning \
  --bucket synapse-data \
  --versioning-configuration Status=Enabled

# Enable server-side encryption
aws s3api put-bucket-encryption \
  --bucket synapse-data \
  --server-side-encryption-configuration '{
    "Rules": [
      {
        "ApplyServerSideEncryptionByDefault": {
          "SSEAlgorithm": "AES256"
        }
      }
    ]
  }'

# Enable MFA delete
aws s3api put-bucket-versioning \
  --bucket synapse-data \
  --versioning-configuration Status=Enabled,MFADelete=Enabled

# Enable logging
aws s3api put-bucket-logging \
  --bucket synapse-data \
  --bucket-logging-status '{
    "LoggingEnabled": {
      "TargetBucket": "synapse-logs",
      "TargetPrefix": "s3-access-logs/"
    }
  }'
```

8.6 Docker Security

8.6.1 Dockerfile Security Best Practices

```
# Use specific base image version, not latest
FROM python:3.11.5-slim-bookworm

# Run as non-root user
RUN groupadd -r appuser && useradd -r -g appuser appuser

# Set working directory
WORKDIR /app

# Copy requirements first for caching
COPY requirements.txt .

# Install dependencies
RUN pip install --no-cache-dir -r requirements.txt

# Copy application code
COPY --chown=appuser:appuser . .

# Switch to non-root user
USER appuser

# Use read-only root filesystem
RUN chmod -R 555 /app

# Health check
HEALTHCHECK --interval=30s --timeout=3s --start-period=5s --retries=3 \
  CMD python -c "import requests; requests.get('http://localhost:8000/health')"

# Don't run as PID 1 (use exec form)
ENTRYPOINT ["python", "manage.py", "runserver", "0.0.0.0:8000"]

# Security labels
LABEL security.scan="required" \
      security.non-root="true" \
      security.read-only-fs="true"
```

8.6.2 Docker Security Scanning

```
# Push image to ECR
aws ecr get-login-password --region us-east-1 | \
  docker login --username AWS --password-stdin
      ACCOUNT.dkr.ecr.us-east-1.amazonaws.com

docker tag synapse:latest ACCOUNT.dkr.ecr.us-east-1.amazonaws.com/synapse:latest

docker push ACCOUNT.dkr.ecr.us-east-1.amazonaws.com/synapse:latest

# Enable image scanning
aws ecr put-image-scanning-configuration \
  --repository-name synapse \
  --image-scanning-configuration scanOnPush=true

# Scan with Trivy
```

```
trivy image --severity CRITICAL,HIGH \
  ACCOUNT.dkr.ecr.us-east-1.amazonaws.com/synapse:latest

# Sign images
cosign sign --key cosign.key \
  ACCOUNT.dkr.ecr.us-east-1.amazonaws.com/synapse:latest

# Verify signature
cosign verify --key cosign.pub \
  ACCOUNT.dkr.ecr.us-east-1.amazonaws.com/synapse:latest
```

8.7 Kubernetes/ECS Task Definition Security

8.7.1 ECS Task Definition

```
{
  "family": "synapse-app",
  "networkMode": "awsvpc",
  "requiresCompatibilities": ["FARGATE"],
  "cpu": "1024",
  "memory": "2048",
  "taskRoleArn": "arn:aws:iam::ACCOUNT:role/synapse-task-role",
  "executionRoleArn": "arn:aws:iam::ACCOUNT:role/synapse-task-execution-role",
  "containerDefinitions": [
    {
      "name": "synapse",
      "image": "ACCOUNT.dkr.ecr.us-east-1.amazonaws.com/synapse:latest",
      "essential": true,
      "portMappings": [
        {
          "containerPort": 8000,
          "protocol": "tcp"
        }
      ],
      "environment": [
        {
          "name": "DJANGO_SETTINGS_MODULE",
          "value": "config.settings.production"
        }
      ],
      "secrets": [
        {
          "name": "DATABASE_PASSWORD",
          "valueFrom":
              "arn:aws:secretsmanager:region:account:secret:synapse/db/password"
        },
        {
          "name": "SECRET_KEY",
          "valueFrom":
              "arn:aws:secretsmanager:region:account:secret:synapse/django/secret-key"
        }
      ],
      "readonlyRootFilesystem": true,
      "mountPoints": [
        {
          "sourceVolume": "tmp",
          "containerPath": "/tmp",
```

```
          "readOnly": false
        }
      ],
      "logConfiguration": {
        "logDriver": "awslogs",
        "options": {
          "awslogs-group": "/ecs/synapse-app",
          "awslogs-region": "us-east-1",
          "awslogs-stream-prefix": "ecs"
        }
      },
      "privileged": false,
      "user": "1000:1000"
    }
  ],
  "volumes": [
    {
      "name": "tmp",
      "emptyDir": {}
    }
  ]
}
```

Chapter 9

Dependency Security

9.1 Python Dependency Scanning

9.1.1 Safety Check

```
# Install safety
pip install safety

# Check dependencies
safety check --json > safety-report.json

# CI/CD integration
safety check --exit-code on-failure
```

9.1.2 Bandit Security Analysis

```
# Install bandit
pip install bandit

# Scan code
bandit -r . -f json -o bandit-report.json

# Scan specific directories
bandit -r synapse/ -ll  # Only report medium+ severity
```

9.1.3 pip-audit Tool

```
# Install pip-audit
pip install pip-audit

# Audit installed packages
pip-audit --desc

# Generate JSON report
pip-audit --format json > audit-report.json
```

9.2 Node.js Dependency Scanning

9.2.1 npm audit

```
# Audit dependencies
npm audit

# Fix vulnerabilities
npm audit fix

# Audit only production dependencies
npm audit --production

# Generate JSON report
npm audit --json > npm-audit.json

# CI/CD: fail on vulnerabilities
npm audit --audit-level=moderate
```

9.2.2 Snyk Integration

```
# Install Snyk
npm install -g snyk

# Test project
snyk test

# Monitor for new vulnerabilities
snyk monitor

# Fix vulnerabilities
snyk fix
```

9.3 Automated Vulnerability Scanning

9.3.1 GitHub Dependabot Configuration

```
# .github/dependabot.yml
version: 2
updates:
  - package-ecosystem: "pip"
    directory: "/"
    schedule:
      interval: "daily"
    allow:
      - dependency-type: "all"
    reviewers:
      - "security-team"
    labels:
      - "security"
    commit-message:
      prefix: "chore: "

  - package-ecosystem: "npm"
```

```yaml
    directory: "/frontend"
    schedule:
      interval: "daily"
    allow:
      - dependency-type: "all"
    reviewers:
      - "security-team"
    labels:
      - "security"

  - package-ecosystem: "docker"
    directory: "/"
    schedule:
      interval: "weekly"
```

9.4 Software Bill of Materials (SBOM)

9.4.1 SBOM Generation

```bash
# Install cyclonedx tools
pip install cyclonedx-bom

# Generate Python SBOM
cyclonedx-bom -o sbom-python.json

# Generate Node.js SBOM
npm install -g @cyclonedx/npm
cyclonedx-npm -o sbom-node.json

# Generate with SPDX format
pip install spdx-tools

# Include in release artifacts
cp sbom-python.json sbom-node.json ./dist/
tar -czf synapse-release-sbom.tar.gz sbom-*.json
```

9.5 Dependency Update Policy

9.5.1 CVE Response SLA

Severity	CVSS Score	Patch Timeline
Critical	9.0-10.0	24 hours
High	7.0-8.9	7 days
Medium	4.0-6.9	30 days
Low	0.1-3.9	90 days

9.5.2 Dependency Pinning

```python
# requirements.txt - Pinned versions
Django==4.2.7
djangorestframework==3.14.0
psycopg2-binary==2.9.9
```

```
redis==5.0.1
celery==5.3.4
fastapi==0.105.0
pydantic==2.5.0

# requirements-dev.txt
pytest==7.4.3
pytest-cov==4.1.0
black==23.12.0
flake8==6.1.0
mypy==1.7.1

# Semantic versioning for updates
# Patch: Bug fixes only (4.2.7 -> 4.2.8)
# Minor: New features, backwards compatible (4.2.x -> 4.3.0)
# Major: Breaking changes (4.x -> 5.0)
```

Chapter 10

Security Monitoring and Incident Response

10.1 Security Logging Requirements

10.1.1 Logging Events

- **Authentication Events**: Login attempts (success/failure), logout, password changes, 2FA enable/disable
- **API Access**: All API requests, user ID, endpoint, method, status code, response time
- **Admin Actions**: User creation, permission changes, system configuration changes
- **Data Access**: Large exports, data downloads, sensitive field access
- **Error Events**: Application errors, database errors, external API failures
- **Security Events**: Failed authorization, suspicious patterns, rate limit violations

10.1.2 Structured JSON Logging

```python
import json
import logging
from pythonjsonlogger import jsonlogger

# Configure JSON logging
logger = logging.getLogger()
handler = logging.StreamHandler()
formatter = jsonlogger.JsonFormatter(
    '%(timestamp)s %(level)s %(message)s %(user_id)s %(ip_address)s'
)
handler.setFormatter(formatter)
logger.addHandler(handler)

# Usage: Log without PII in message
logger.info(
    'User login successful',
    extra={
        'timestamp': datetime.now().isoformat(),
        'level': 'INFO',
        'user_id': user.id,
        'ip_address': get_client_ip(request),
```

```
        'user_agent': request.META.get('HTTP_USER_AGENT'),
    }
)

# Log format:
# {"timestamp": "2024-01-15T10:30:45.123Z", "level": "INFO",
#  "message": "User login successful", "user_id": 12345,
#  "ip_address": "203.0.113.42"}
```

10.2 Log Aggregation with CloudWatch

```
# settings.py
LOGGING = {
    'version': 1,
    'disable_existing_loggers': False,
    'formatters': {
        'json': {
            '()': 'pythonjsonlogger.jsonlogger.JsonFormatter',
            'format': '%(asctime)s %(name)s %(levelname)s %(message)s'
        }
    },
    'handlers': {
        'watchtower': {
            'level': 'INFO',
            'class': 'watchtower.CloudWatchLogHandler',
            'log_group': '/aws/synapse/application',
            'stream_name': 'app-logs',
            'formatter': 'json',
        },
    },
    'root': {
        'handlers': ['watchtower'],
        'level': 'INFO',
    },
}
```

10.3 SIEM Integration: ELK Stack

10.3.1 Logstash Configuration

```
input {
  syslog {
    port => 514
    type => "syslog"
  }

  cloudwatch_logs {
    log_group_name => "/aws/synapse/application"
    start_position => "end"
  }
}

filter {
  if [type] == "syslog" {
```

```
    grok {
      match => { "message" => "%{SYSLOGLINE}" }
    }
  }

  # Parse JSON logs
  json {
    source => "message"
  }

  # Remove PII
  mutate {
    remove_field => ["email", "password_hash", "credit_card"]
  }

  # Geolocation enrichment
  geoip {
    source => "ip_address"
  }
}

output {
  elasticsearch {
    hosts => ["elasticsearch.internal:9200"]
    index => "synapse-logs-%{+YYYY.MM.dd}"
  }

  # Alert on critical events
  if [level] == "CRITICAL" {
    email {
      to => "security@synapse.io"
      subject => "CRITICAL␣SECURITY␣EVENT"
    }
  }
}
```

10.4 Intrusion Detection

10.4.1 fail2ban Configuration

```
# /etc/fail2ban/jail.local
[DEFAULT]
bantime = 3600
findtime = 600
maxretry = 5

[sshd]
enabled = true
port = ssh
filter = sshd
maxretry = 3

[django-auth]
enabled = true
port = http,https
filter = django-auth
logpath = /var/log/synapse/django.log
```

```
maxretry = 5
bantime = 1800

[nginx-http-auth]
enabled = true
port = http,https
filter = nginx-http-auth
logpath = /var/log/nginx/error.log
```

10.4.2 AWS GuardDuty

```
# Enable GuardDuty
aws guardduty create-detector --enable --region us-east-1

# List findings
aws guardduty list-findings --detector-id DETECTOR_ID

# Create SNS notification for findings
aws events put-rule \
  --name guardduty-findings \
  --event-pattern '{"source":["aws.guardduty"]}'

aws events put-targets \
  --rule guardduty-findings \
  --targets "Id"="1","Arn"="arn:aws:sns:us-east-1:ACCOUNT:security-alerts"
```

10.5 Alert Thresholds

10.5.1 Alert Configuration

Metric	Threshold	Action
Failed logins per IP	¿10/min	Ban IP for 30 min
Failed logins per user	¿5/hour	Lock account for 1 hour
API errors	¿5% error rate	Page on-call
Unusual API patterns	¿1000 req/min from single IP	Rate limit + alert
Database query time	¿5000ms average	Alert + slow query log
PII field access	Any access to phone/SSN	Log + audit

10.6 Incident Response Plan

10.6.1 Incident Response Phases

1. **Detection**: Automated alerts trigger incident creation
2. **Containment**: Isolate affected systems, revoke credentials, block attackers
3. **Eradication**: Remove attacker access, patch vulnerabilities

4. **Recovery**: Restore systems, monitor for re-infection
5. **Lessons Learned**: Post-mortem, process improvements

10.6.2 Incident Severity and SLA

Level	Description	Response Time
P1 Critical	Data breach, service down, RCE	1 hour
P2 High	Unauthorized access, credential compromise	4 hours
P3 Medium	Privilege escalation attempt, config issue	24 hours
P4 Low	Suspicious but contained, info disclosure	7 days

10.6.3 Incident Response Playbook

```
P1 CRITICAL INCIDENT RESPONSE
1. Alert triggered -> Incident created in Jira
2. On-call engineer paged immediately (PagerDuty)
3. Incident commander assumes leadership role
4. Affected systems isolated from network
5. Evidence collected for forensics
6. AWS root account security keys rotated
7. All active sessions terminated
8. Backup systems activated
9. External communications: status.synapse.io updated
10. Post-incident analysis within 24 hours

P2 HIGH INCIDENT RESPONSE
1. Alert triggered -> Incident created
2. On-call engineer notified
3. Root cause analysis initiated
4. Affected user accounts reviewed
5. Logs analyzed for scope of breach
6. Security patches applied
7. Response time: 4 hours to initial containment
```

10.7 Vulnerability Disclosure Policy

10.7.1 Security.txt

```
# /.well-known/security.txt
Contact: security@synapse.io
Expires: 2025-12-31T23:59:59.000Z
Preferred-Languages: en
Canonical: https://synapse.io/.well-known/security.txt
Policy: https://synapse.io/security/responsible-disclosure
Acknowledgments: https://synapse.io/security/thanks
```

10.7.2 Responsible Disclosure Process

1. Security researcher discovers vulnerability
2. Reports to security@synapse.io with details
3. We acknowledge receipt within 24 hours
4. Investigation period: 7-30 days depending on severity
5. Researcher may request CVE allocation
6. Patch released with public advisory
7. Researcher credited publicly (if desired)

Chapter 11

Penetration Testing Checklist

11.1 Reconnaissance Tests

1. DNS enumeration: nslookup, dig, zone transfers
2. Subdomain discovery: subfinder, amass
3. Technology identification: whatruns, builtwith
4. WHOIS information gathering
5. Shodan/Censys scanning for exposed services
6. Git repository history analysis
7. Public code repository scanning (GitHub, GitLab)
8. S3 bucket discovery and enumeration
9. Infrastructure mapping: port scanning, service detection

11.2 Authentication Bypass Tests

1. Default credentials on admin panels
2. SQL injection on login form
3. JWT token manipulation: expiration, signature bypass
4. Session fixation attacks
5. Password reset token prediction
6. Email verification bypass
7. OAuth2/OIDC flow exploitation
8. Brute force attacks with rate limit bypass
9. Credential stuffing with common password lists

11.3 Authorization Tests (IDOR & Privilege Escalation)

1. Direct object reference manipulation (IDs in URLs)
2. Cross-user data access attempts
3. Privilege escalation through API parameters
4. Role manipulation attacks
5. Admin panel bypass attempts
6. Permission boundary testing
7. Object ownership verification
8. Function-level authorization testing

11.4 Injection Tests

1. SQL Injection: Single quote, UNION, time-based, blind
2. NoSQL Injection: MongoDB, Redis query injection
3. Command Injection: Shell metacharacters, pipe operators
4. SSTI (Server-Side Template Injection): Jinja2, Mako
5. XSS (Cross-Site Scripting): Stored, reflected, DOM-based
6. LDAP Injection: Active Directory queries
7. XML/XPath Injection: XML external entity attacks
8. Log Injection: Newline insertion for log spoofing

11.5 API Security Tests

1. Rate limiting bypass (header manipulation, IP rotation)
2. Broken object level authorization
3. Mass assignment attacks
4. API key exposure in responses
5. Unencrypted sensitive data in API responses
6. GraphQL injection and enumeration
7. API versioning bypass
8. Webhook security (signature verification, replay attacks)
9. API documentation exposure

11.6 Business Logic Tests

1. Price manipulation in shopping carts
2. Workflow bypass (skipping required steps)
3. Race conditions in critical operations
4. Insufficient input validation on business logic
5. Transaction reversal attacks
6. State manipulation attacks
7. Negative quantity/amount acceptance
8. Duplicate transaction processing

11.7 Infrastructure Tests

1. AWS S3 bucket misconfiguration
2. CloudFormation template exposure
3. EC2 security group overly permissive rules
4. RDS snapshots with public access enabled
5. Lambda function URL public access
6. Secrets Manager access from unexpected roles
7. CloudWatch Logs exposure
8. VPC Flow Logs disclosure
9. EBS snapshot sharing across accounts

11.8 Penetration Testing Tools

- **OWASP ZAP**: Automated web application scanner
- **Burp Suite Professional**: Manual testing proxy
- **sqlmap**: SQL injection detection and exploitation
- **nikto**: Web server vulnerability scanner
- **nmap**: Network reconnaissance and port scanning
- **Metasploit**: Exploitation framework
- **john**: Password cracking tool
- **hashcat**: GPU-accelerated password cracking
- **hydra**: Online password guessing tool
- **wireshark**: Network traffic analysis

11.9 Testing Frequency

- **Automated Scanning**: Continuous (weekly)
- **Manual Penetration Testing**: Quarterly (4x per year)
- **Major Release Testing**: Before each production release
- **Critical Patch Testing**: After critical security patches
- **Annual Assessment**: Full security assessment by third party

Chapter 12

GDPR and Privacy Compliance

12.1 Data Subject Rights

12.1.1 Right of Access

Users can request and receive a copy of all personal data we hold about them in a portable format (CSV, JSON).

12.1.2 Right to Rectification

Users can request correction of inaccurate personal data. We provide self-service tools to update their profile.

12.1.3 Right to Erasure (Right to be Forgotten)

Users can request deletion of their data. We soft-delete accounts and permanently delete after 30 days.

12.1.4 Right to Data Portability

Users can download their data in standardized format (JSON/CSV) for transfer to another service.

12.1.5 Right to Restrict Processing

Users can request suspension of processing while they contest data legality.

12.1.6 Right to Object

Users can object to marketing communications and profiling.

12.2 Privacy by Design

1. Minimal data collection: Only essential data collected
2. Data minimization: Delete data no longer needed
3. Purpose limitation: Use data only for stated purposes
4. Storage limitation: Retention policies enforced automatically
5. Integrity and confidentiality: Encryption and access control
6. Accountability: Audit logs and documentation maintained

12.3 Cookie Consent

12.3.1 Cookie Banner Implementation

- Visible banner on first visit
- Explicit consent required before setting cookies
- Granular consent options: Analytics, marketing, necessary
- Consent recorded and timestamped
- Ability to withdraw consent at any time

12.3.2 Cookie Categories

Type	Purpose
Necessary	Session management, CSRF protection, security
Functional	Language preferences, UI customization
Analytics	Google Analytics, user behavior tracking
Marketing	Remarketing ads, conversion tracking

12.4 Data Processing Agreement (DPA)

A DPA template includes:

1. Scope: Data processed, duration, nature
2. Subject Matter: Type of processing (e.g., user accounts)
3. Duration: How long data is processed
4. Nature: What happens to data
5. Purpose: Why data is processed
6. Processor Obligations: Security, confidentiality, sub-processors
7. Data Subject Rights: Support for GDPR requests
8. Security Measures: Technical and organizational safeguards
9. Sub-processors: List of third-party processors
10. International Transfers: Mechanisms (SCCs, BCRs) if applicable

12.5 Privacy Policy Requirements

The privacy policy must clearly specify:

1. Data controller identity and contact
2. Legal basis for processing
3. Data categories collected
4. Processing purposes
5. Data retention periods
6. Data subject rights and how to exercise them
7. Complaint procedure to data protection authority
8. Automated decision-making and profiling (if applicable)
9. Third-party sharing
10. International data transfers

12.6 Data Breach Notification

12.6.1 72-Hour Rule

Data breaches must be reported to the supervisory authority within 72 hours of discovery.

12.6.2 Breach Notification Process

1. Incident detected -¿ Immediate investigation
2. Determine scope: What data was accessed?
3. Assess risk: Would affected users be harmed?
4. Document findings with timestamps
5. If high risk: Notify affected users immediately
6. If any affected: Notify supervisory authority within 72 hours
7. If public interest: Public disclosure may be required
8. Record all notifications and responses

12.7 Data Protection Officer (DPO)

12.7.1 DPO Responsibilities

- Monitor GDPR compliance
- Serve as contact point for data subjects and authorities
- Conduct privacy impact assessments
- Maintain documentation of processing activities
- Provide privacy training to staff

- Investigate complaints and breaches
- Report to the data protection authority

Chapter 13

Security Audit Checklist

13.1 Pre-Deployment Security Checklist (50+ Items)

13.1.1 Code Security

1. No hardcoded secrets (API keys, passwords, tokens)
2. No debug mode enabled (DEBUG=False)
3. No test accounts with weak passwords
4. SQL injection prevention: Parameterized queries used
5. XSS prevention: Input sanitization and output encoding
6. CSRF protection: CSRF tokens on all state-changing endpoints
7. Authentication: Strong password requirements enforced
8. Authorization: Permission checks on all endpoints
9. Input validation: All inputs validated and sanitized
10. Error handling: Errors logged without exposing sensitive info
11. Logging: All sensitive operations logged
12. Dependencies: All dependencies security-scanned
13. Code review: Minimum 2 reviewers approved
14. Static analysis: SAST tool passed without critical/high issues

13.1.2 Infrastructure Security

1. SSL/TLS: TLS 1.3 enabled, valid certificate
2. Database: Not publicly accessible
3. Database: Encrypted at rest with KMS
4. Database: SSL connections required
5. Database: Least privilege users configured
6. S3: No public ACLs or bucket policies

7. S3: Versioning enabled
8. S3: Encryption enabled
9. EC2: Security groups follow least privilege
10. EC2: No SSH from internet
11. VPC: Database in private subnet
12. VPC: VPC Flow Logs enabled
13. IAM: No root account usage
14. IAM: MFA enabled on root account

13.1.3 Application Security

1. HSTS: Strict-Transport-Security header set
2. CSP: Content-Security-Policy header configured
3. X-Frame-Options: Set to DENY
4. X-Content-Type-Options: Set to nosniff
5. XSS-Protection: Header set
6. Referrer-Policy: Configured appropriately
7. CORS: Whitelist-only origins
8. Rate limiting: Configured and tested
9. Session: HTTP-only, secure, SameSite cookies
10. API keys: Properly generated and rotated
11. Secrets: In Secrets Manager, never in code
12. Logging: No PII in logs
13. Monitoring: CloudWatch/ELK configured
14. Alerts: Alert rules configured for security events

13.1.4 Compliance

1. GDPR: Privacy policy published
2. GDPR: Consent mechanism functional
3. GDPR: Right to erasure implemented
4. GDPR: Data portability implemented
5. PII: Minimal collection enforced
6. PII: Encryption at rest
7. PII: No unencrypted transmission
8. Security.txt: Published at /.well-known/security.txt

9. Terms of Service: Published and current
10. Data Processing Agreement: Signed with vendors
11. Audit logging: Enabled and tested
12. Incident response: Plan documented and communicated

13.2 Code Review Security Checklist

1. Secrets not committed (AWS credentials, API keys, etc.)
2. No SQL injection vulnerabilities
3. No XSS vulnerabilities
4. CSRF tokens present and validated
5. Authentication and authorization correct
6. Error messages don't leak sensitive information
7. Input validation on all user-supplied data
8. Output encoding for template rendering
9. Cryptography implemented correctly (no custom crypto)
10. Dependencies are secure versions
11. Race conditions or TOCTOU issues
12. File upload validation (type, size, content)
13. Deserialization safe (no pickle of untrusted data)
14. Logging doesn't expose sensitive data
15. Comments don't contain security information
16. Code complexity understandable and secure

13.3 Quarterly Security Review Tasks

1. Review and rotate AWS credentials
2. Update dependency vulnerability reports
3. Review CloudWatch logs and dashboards
4. Analyze failed authentication attempts
5. Review admin access logs
6. Check SSL/TLS certificate expiration
7. Audit IAM permissions for least privilege
8. Review security group rules
9. Check for unpatched CVEs in dependencies

10. Update threat model based on recent attacks
11. Security awareness training for team
12. Review incident response plan
13. Scan codebase with SAST tools
14. Test backup restoration
15. Review third-party processor security

13.4 Annual Requirements

1. Full penetration test by third party
2. Security code review by external auditors
3. GDPR compliance audit
4. Disaster recovery and backup testing
5. Security training for all staff
6. Update and test incident response plan
7. Review and update threat model
8. Architecture security review
9. Review of security incidents and lessons learned
10. Update of security documentation
11. Risk assessment across all systems

Chapter 14

Security Training and Culture

14.1 Developer Security Training

14.1.1 Required Topics

1. OWASP Top 10: Recognition and prevention
2. Secure coding practices in Python and JavaScript
3. Django security features and best practices
4. FastAPI security implementation
5. SQL injection prevention and parameterized queries
6. XSS prevention and input validation
7. CSRF protection mechanisms
8. Authentication and session management
9. Cryptography basics and usage
10. API security principles
11. Database security
12. Infrastructure security (AWS, Docker)
13. Security incident response
14. Responsible disclosure

14.1.2 Training Schedule

- **Onboarding**: 4-hour security fundamentals course
- **Quarterly**: 1-hour security topics deep dive
- **Annual**: Full-day security training refresh
- **As needed**: Specialized training for new technologies

14.2 Secure Coding Guidelines

14.2.1 Golden Rules

1. Never trust user input: Validate and sanitize everything
2. Use frameworks: Don't implement security yourself
3. Fail securely: Deny access by default
4. Defense in depth: Multiple layers of security
5. Keep it simple: Complexity breeds vulnerability
6. Use established libraries: Never roll custom crypto
7. Log and monitor: Detect attacks in progress
8. Assume breach: Design for zero trust
9. Update regularly: Security patches matter
10. Test continuously: Security tests as part of CI/CD

14.2.2 Code Example: Secure vs. Insecure

```python
# INSECURE
def get_user(user_id):
    query = f"SELECT * FROM users WHERE id = {user_id}"
    return db.execute(query)

# SECURE
def get_user(user_id):
    user = User.objects.get(id=user_id)
    if user != request.user and not request.user.is_staff:
        raise PermissionDenied()
    return user

# INSECURE
password = request.POST.get('password')
hashed = hashlib.sha256(password).hexdigest()

# SECURE
from django.contrib.auth.hashers import make_password
password = request.POST.get('password')
hashed = make_password(password)  # Uses bcrypt
```

14.3 Security Champions Program

14.3.1 Program Overview

- Identify security champions from each team
- Provide advanced security training
- Champions review code for security issues
- Champions lead security discussions in standups

- Champions report security findings to security team
- Quarterly champions meeting for knowledge sharing
- Recognition and incentives for security contributions

14.3.2 Champion Responsibilities

1. Review team code for security vulnerabilities
2. Mentor team members on secure coding
3. Participate in security training delivery
4. Lead threat modeling discussions
5. Report security concerns to security team
6. Stay updated on security best practices
7. Represent security interests in architecture discussions
8. Help with incident response exercises

14.4 Security Documentation

All security practices should be documented:

- **Security Architecture Document**: System design and security controls
- **Threat Model**: Assets, threats, mitigations
- **Security Runbooks**: Step-by-step procedures for common tasks
- **Incident Response Plan**: Playbooks for different incident types
- **DRP/BCP**: Disaster recovery and business continuity plans
- **Policy Documents**: Password policy, access control policy, etc.
- **Audit Logs**: Retained for compliance and forensics

Appendix A

Additional Resources

A.1 External References

- OWASP Top 10 2021: https://owasp.org/Top10/
- NIST Cybersecurity Framework: https://www.nist.gov/cyberframework
- CWE Top 25: https://cwe.mitre.org/top25/
- AWS Security Best Practices: https://aws.amazon.com/security/best-practices/
- Django Security Documentation: https://docs.djangoproject.com/en/4.2/topics/security/
- GDPR Official Text: https://gdpr-info.eu/
- ISO 27001:2022: https://www.iso.org/standard/27001

A.2 Tools and Resources

- OWASP ZAP: https://www.zaproxy.org/
- Burp Suite: https://portswigger.net/burp
- Snyk: https://snyk.io/
- Dependabot: https://dependabot.com/
- Safety: https://pyup.io/safety/
- Bandit: https://bandit.readthedocs.io/